

Tips for practical 2 test

- **2 question with sub questions**
- Overall 2 questions
- **Question1** covered **chapter 4 (7 questions)** and **chapter 5 (3 questions)**
- **Question 2** covered **chapter 5 (shell script)**, given a figure then you need to write a script based on that figure. Either you need **calculate triangle or total payment or calculate rectangle**
- Today we are going to create a simple script to multiply 2 number, then save the result in another file using >> operator, wch8ih means you append the results to another file
- For calculation you need to use let command(+-* /), do not use expr command(*) because expr command can only read the value you pass in using variable but cannot store the value

Section A

Identify the command for all the question below:

1. How to display the value that is stored inside the **MYNAME** variable.
`echo "$MYNAME"`
2. Create a variable "**NUM**" and assign the value "**77**" to this variable
`NUM=77`
3. How to clear the content of the variable "**NAME**"
`unset NAME`
4. How to list the content of **/home** directory and **/root** directory to a file named "**Hello.txt**" and any standard error output is saved to another file named "**Bin.txt**"
`ls /home /root > Hello.txt 2>Bin.txt`

Redirection is used to specify input, output and error standard in linux.

Output redirection using > operator

Input redirection is < operator

Error standard is using 2> operator

Output redirection example:

ls /home/user > list.txt

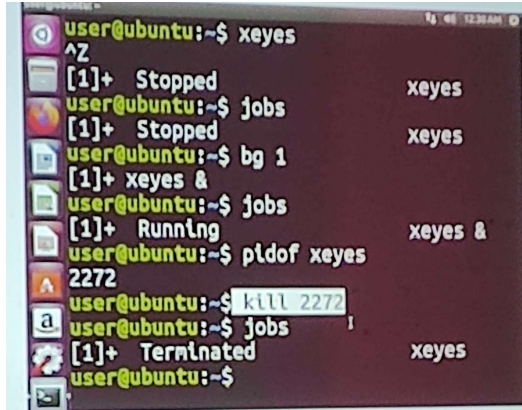
- **Meaning you are going to list all file and directory into list.txt file, before this if you ls /home/user meaning you list all files and directory into your screen**

Error standard example:

ls /home/user > list.txt 2> bin.txt

- **Meaning you are going to list all files and directory into list.txt and any error happen during listing, you are going to put all the error result into bin.txt (purpose is don't want to mix the actual result with error result)**

5. List all processes in the form of tree structure, then terminate the first process which has PID of 2023.
`pstree`
`kill 2023`



```
user@ubuntu:~$ xeyes
^Z
[1]+  Stopped                  xeyes
user@ubuntu:~$ jobs
[1]+  Stopped                  xeyes
user@ubuntu:~$ bg 1
[1]+  xeyes &
user@ubuntu:~$ jobs
[1]+  Running                  xeyes &
user@ubuntu:~$ pkill xeyes
2272
user@ubuntu:~$ kill 2272
user@ubuntu:~$ jobs
[1]+  Terminated              xeyes
user@ubuntu:~$
```

6. Search for any file bigger than 70k regardless of the user's connection to the terminal and save the search results to a file called **ResultLog.txt**
`nohup find -size +70k > ResultLog.txt`

“nohup” command is used to run the process even though you close the terminal or log out from system (you let the process run even though you close the terminal)

“nohup” used to run the process even though user log out from system

“find” is to search the file

“size” is used to declare the file size

-> redirection is used to store all the searching result into a file, in this case ResultLog.txt

7. Execute a background process (&) that displays a reminder to submit assignment in 1 minute's time.

`sleep 60 ; echo "Reminder to submit assignment" &`

8. By using cron, schedule the script in **/home/user** directory named **Remainder** to run from Monday to Friday at 11AM.x

`crontab -e` (create a script if does not yet exist / edit the existing script)

`0 11 * * 1-5 /home/user/Remainder` (create a file called Remainder in home user according the date and time asked)

9. Change the priority of the **xeyes** program which has PID of 1234 from a default nice value (+10) to a nice value of **-5**.

`renice -5 1234`

10. Display the contents of a file called “output” with the letters a , b, and c changed to A, B and C respectively by using output redirection “>” **“tr = translate”**

`-tr abc ABC > output`

Redirection is used to specify input, output and error standard in linux.

- Output redirection using > operator

- Input redirection is using < operator
- Error standard is using 2> operator
-
- Output redirection example:
- `ls /home/user > list.txt`
- Meaning you are going to list all files and directory into List.txt file, before this if you `ls /home/user` meaning you list all files and directory into your screen
-
- Error standard example:
- `ls /home/user > list.txt 2>bin.txt`
- Meaning you are going to list all files and directory into List.txt and any error happen during listing, you are going to put all the error result into bin.txt (purpose is don't want to mix the actual result with error result)
-

Section B

- a) You are required to create a shell script named **calculateAreaofTriangle.sh**. The script will allow the user to enter the height and base of a triangle. Then the script will calculate the area of the triangle and display it to the user.

Area of Triangle = height * base / 2

The user's input and results of the calculation are then written into a text file name Result.txt under your user's home directory. After storing the user's input and results into the text file, ask the user if they want to continue. If the user input 'Y', prompt the user for the height and base again.

- use let command to calculate the area

```
nano calculateAreaofTriangle.sh
```

```
echo "=====
```

```
echo "Calculate the area of triangle"
```

```
echo "=====
```

```
AGAIN = "Y"
```

```
while [ "$AGAIN" = "Y" ]
```

```
do
```

```
    echo -n "enter the height: "
```

```
    read height
```

```
    echo -n "enter the base: "
```

```
    read base
```

```
    let AREA=$height*$base/2
```

```
    echo "the height is" $height
```

```
    echo "the base is: $base"
```

```
    echo "the area is $AREA"
```

```
    echo "Height: $height, Base: $base, AREA: $AREA">>Result.txt
```

```
    echo -n "Do you want to continue? Y/N "
```

```
    read AGAIN
```

```
done
```

```
while [ $confirm = "yes" ]
```

```
do
```

```
    read -p "Enter the height: " height
```

```
    read -p "Enter the base: " base
```

```
    area=$((height * base / 2))
```

```
    echo "Area is $area" >> Result.txt
```

```
    read -p "Do you wish to continue: " confirm
```

```
done
```

- b) You are required to create a shell script named **calculateCharger.sh**. The script will allow the user to enter the number of tickets and calculate the total payment using the formula below.

(total= number of tickets * RM10)

Note: Assume that the ticket price is RM10

The user's input and results of the calculation are then written into a text file named **payment.txt** under your user's home directory, after storing the user's input and results into the text file, ask the user if they want to continue. If the user input "Y", prompt the user for the number of ticket again.

```
nano calculateCharger.sh
```

```
echo "=====
```

```
echo "Calculate Total Payment"
```

```
echo "=====
```

```
AGAIN = "Y"
```

```
while [ "$AGAIN" = "Y" ]
```

```
Do
```

```
    echo -n "Enter number of tickets: "
```

```
    read t
```

```
    let Total= $t*10
```

```
    echo "Number of tickets: $t"
```

```
    echo "The total payment is $Total"
```

```
    Total payment: $Total">>~/payment.txt
```

```
    echo -n "Do you want to continue? Y/N "
```

```
    read AGAIN
```

```
done
```

```
while [ $confirm = "yes" ]
```

```
do
```

```
    read -p "Enter number of crickets: " crickets
```

```
    total=$((crickets * 10))
```

```
    echo "Number of tickets : $crickets
```

```
    Total Price : RM $total " >> payment.txt
```

```
    read -p "Do you wish to continue: " confirm
```

```
done
```